Quantized Guided Pruning for Efficient Hardware Implementations of Deep Neural Networks

Ghouthi BOUKLI HACENE*†, Vincent GRIPON†, Matthieu ARZEL†, Nicolas FARRUGIA† and Yoshua BENGIO*
*IMT Atlantique, Lab-STICC
†Université de Montréal, MILA

***Abstract*—Deep Neural Networks (DNNs) in general and Convolutional Neural Networks (CNNs) in particular are state-of-the-art in numerous computer vision tasks such as object classification and detection. However, the large amount of parameters they contain leads to a high computational complexity and strongly limits their usability in budget-constrained devices such as embedded devices. In this paper, we propose a combination of a pruning technique and a quantization scheme that effectively reduce the complexity and memory usage of convolutional layers of CNNs, by replacing the complex convolutional operation by a low-cost multiplexer. We perform experiments on CIFAR10, CIFAR100 and SVHN datasets and show that the proposed method achieves almost state-of-the-art accuracy, while drastically reducing the computational and memory footprints compared to the baselines. We also propose an efficient hardware architecture, implemented on Field Programmable Gate Arrays (FPGAs), to accelerate inference, which works as a pipeline and accommodates multiple layers working at the same time to speed up the inference process. In contrast with most proposed approaches which have used external memory or software defined memory controllers, our work is based on algorithmic optimization and full-hardware design, enabling a direct, on-chip memory implementation of a DNN while keeping close to state of the art accuracy.**

I. Introduction and Related Work

For the past few years, Deep Neural Networks (DNNs), and especially Convolutional Neural Networks (CNNs) [1], have received considerable attention thanks to their remarkable accuracy in computer vision tasks [2]–[5]. However, the need for intensive computations and memory leaded to the fact most DNN implementations are based on GPUs. Consequently, providing efficient hardware implementations is a very active and prospective field of research. Therefore, the deployment of CNNs in embedded systems is complex and potentially blocking for many applications.

To address this limitation, multiple approaches have been proposed. For example, in [6] the authors have introduced Binary Connect (BC), a quantization method that aims at binarizing the weights during the learning phase. As a result, the obtained DNN contains only weights whose values are 1 or -1, while suffering from a very limited drop in accuracy compared to full precision baselines. These works have been improved later in [7], where the authors introduce Binary Weight Network (BWN), and propose to add a scaling factor per layer and per kernel, as a mean to offer better diversity to binary networks, with almost no impact on memory usage or complexity. Other approaches have proposed to limit weights to three or more values (-1, 0, and 1) [8], [9]. These approaches have demonstrated that using slightly more bits to encode weights enable to improve accuracy by a significant amount, but they also require much more memory and other hardware components to compute non-binary operations.

Once a binary neural network has been trained, efficient implementations can benefit from simplified operations. For example, multiplications in binary neural networks can be replaced by simple low-cost multiplexers. Efficient solutions have been proposed in [10], [11]. However, even binary neural networks still require significant computational power and memory. These solutions also typically lead to a considerable latency, which may be an issue for some applications.

In other approaches, authors have aimed at reducing the number of DNNs parameters using pruning method. For example, Wu et al. [12] introduced Shift Layers (SLs), a substitution to convolutional layers, in which the complex convolution operation is replaced by a shift operation followed by a multiplication. In [13], the authors introduced Shift Attention Layers (SALs), a novel and efficient structured pruning method that aims at replacing convolutional layers by shift layers, with a notable difference which is that shift operations are learned during training phase contrary to the basic approach introduced in [12] where shifts are handcrafted.

Another line of work relies on implementing DNNs architectures and accelerating inference phase on embedded systems with limited resources such as FPGAs. For instance, in [14]–[16] the authors attempted to accelerate CNNs on FPGA and achieved a considerable high frame rate. However, such hardware implementations show an important drop in accuracy comparing to state-of-the-art. In [17], the authors proposed a hardware architecture to deploy VGG16 on FPGA with a good accuracy. However, they ended up with a low frame rate for a real time image classification. Recently, Yang et al [18] proposed a shift layer based hardware implementation to accelerate CNNs on FPGA while keeping an acceptable accuracy. However, the authors considered only the processing blocks for a hardware implementation and assumed that data is stored on an external memory, and different processing blocks are controlled using an external CPU, and which makes pipeline more difficult to implement, and thus speeding up data processing more challenging.

In this paper, we propose to combine SAL with binary quantization of weights using BC or BWN, resulting in very

978-1-7281-7044-2/20/$31.00 ©2020 IEEE
Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 06:46:50 UTC from IEEE Xplore. Restrictions apply.

lightweight DNNs in which complex convolution operations are replaced by low cost multiplexers that considerably eases hardware implementation on FPGA. We show in the following that such a combination approaches state-of-art accuracy while reducing memory footprint and computational power of DNNs. We also propose a hardware architecture in which we consider all processing blocks, memory blocks and controllers required, and implement such an architecture which uses very few resources and computational power on an FPGA, without using any external resources. This implementation can compute more than one layer at a time and uses a simple multiplexer to replace convolutional operations and process data through DNN layers. As such, it provides significantly smaller latency than existing counterparts, as shown in Section III.

II. Shift Layers (SLs) and Shift Attention Layers (SALs)

Let us denote by $\mathbf{x}$ (resp. $\mathbf{y}$ or $\mathbf{w}$) the input (resp. output or kernel) tensor of a given convolutional layer. We index $\mathbf{x}$ (resp. $\mathbf{y}$) using three indices i, j, k (resp. ℓ), where $0 \leq i < i_{\max}$ and $0 \leq j < j_{\max}$ correspond to 2D coordinates and $0 \leq k < k_{\max}$ (rsp. $0 \leq \ell < \ell_{\max}$) indexes a feature map. Similarly, we index $\mathbf{w}$ using four indices: $0 \leq \iota \leq \iota_{\max}$ and $0 \leq \lambda \leq \lambda_{\max}$ correspond to 2D coordinates, and k and ℓ are as introduced above. So, an element of the input tensor is written $x_{i,j,k}$, an element of the kernel tensor is written $w_{\iota,\lambda,k,l}$ and an element of the output tensor is written $y_{i,j,l}$.

To obtain an SL instead of Convolutional Layer (CL), the authors in [12] propose to remove most of the connections in each slice $\mathbf{w}_{\cdot,\cdot,k,\ell}$ of the kernel tensor at the initialisation process. The connections to be kept are chosen according to a deterministic rule agnostic of the initialization and of the training dataset. Namely, the authors choose to only keep the connections $w_{\iota,\lambda,k,\ell}$ for which

$$\iota + \lambda\iota_{\max} = k \pmod{\iota_{\max}\lambda_{\max}}. \quad (1)$$

When considering 3×3 kernels, 89% of the connections are pruned in the convolutional layer. Then, the training process is performed on remaining connections, disregarding the other ones. Using such a configuration, all connections in a 3×3 kernel are pruned but one, and thus the convolution of each slice of the kernel tensor is replaced by a simple multiplication.

Contrary to SL where the kept connection is predetermined, SAL uses an attention mechanism [19] that selects the most relevant connection for each kernel and prunes the others. This process is performed during training, so that at the end of the training phase, the network configuration corresponds exactly to one obtained using SL.

To further benefit from the reduced complexity of the SAL method, we combine it with a weight binarization method such as binary connect (BC) or binary weight network (BWN). Once remaining connections have been binarized, it is possible to replace the multiplication operation by a multiplexer, and thus, such a combination requires only low cost multiplexers to process data during inference instead of computing complex convolutional operations.

Results

To evaluate the performance of our proposed combination, we use the CIFAR10 vision benchmark. We compare various modern CNN architectures such as Resnet [20], Wide-Resnet [21] and Densenet [22]. Note that these architectures contain 1×1 and 3×3 convolutional kernels only. Thus we apply the proposed method on the 3×3 kernels.

We report in Table I the obtained results when using Equation (1) to remove kernels connections (SL), when applying SAL, when applying binarization methods, and when combining shift layers with binarization, and compare the accuracy obtained with baseline architectures. Note that BC and BWN offers a 32 compression factor in terms of memory used, and SL or SAL method roughly multiply this factor by 9, achieving an almost 300 factor compression in total. Interstingly, we observe that in the case of using SAL with BWN, the obtained accuracy remains at most 1% away to that of the baseline. We also perform experiments on SVHN (resp. CIFAR100) on Resnet18 and obtain 97%/96% (resp. 78%/75.2%) accuracy for Full-precision/SAL+BWN.

III. Hardware Implementation

In Figure 1, we depict the proposed hardware architecture to perform the combination of SAL and BC (or BWN), which we name a "layer block". This architecture uses a simple low-cost multiplexer. In more details, a layer block is made of two sub-blocks: a memory one and a processing unit one.

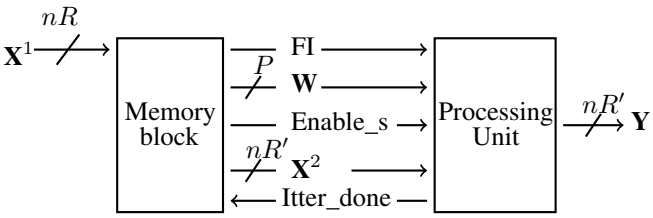


Fig. 1. Hardware architecture of a layer block.

The memory block contains two block RAMs (BRAMs) where content is encoded using n bits fixed point. The first is used to store the computed feature maps. Once they are all computed, the content of the first BRAM is copied to the second one, so that it becomes the input of the next layers. At the same time, the computed feature maps of another image can be stored in the first BRAM. We thus obtain a pipeline architecture, in which all implemented layers work at the same time to speed up the classification process.

To avoid data overflow, we process each row of a slice of the input tensor $\mathbf{x}$ independently, and each slice of the kernel tensor independently as well. In more details, we copy from BRAM one to BRAM two a feature subvector $\mathbf{X}^2_{i,k} = \{x^2_{i,1,k}, x^2_{i,2,k}, \ldots, x^2_{i,R',k}\}$ made of R' values, instead of the whole feature vector $\mathbf{X}^1_{i,k} = \{x^1_{i,1,k}, x^1_{i,2,k}, \ldots, x^1_{i,R,k}\}$ made of $R > R'$ values (cf. Figure 1). This is to account for the border effects (padding). To simplify notations, we replace $\mathbf{X}^1_{i,k}$ (resp. $\mathbf{X}^2_{i,k}$) by $\mathbf{X}^1$ (resp. $\mathbf{X}^2$) in the following.

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 06:46:50 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF ACCURACY BETWEEN BASELINE ARCHITECTURES, PRUNED ONES, BINARIZED ONES, AND THE PROPOSED METHOD ON CIFAR10.

	Resnet18	Resnet34	WideResnet-28-10	Densenet121
Full-precision	94.5%	95%	96.2%	95%
SL	93.5%	93.8%	95%	94.3%
SAL	94.2%	94.9%	96%	94.8%
BC	93.31%	93.64%	95.2%	94.5%
BWN	94.6%	94.9%	95.9%	94.92%
SL + BC	91%	91.3%	94%	93%
SAL + BWN	93.5%	94.6%	95.4%	94.6%

TABLE II
FPGA RESULTS FOR THE PROPOSED ARCHITECTURE ON VU13P (XCVU13P-FIGD2104-1-E).

	P	LUT	FF	BRAMs	Frequency	Processing Latency	Processing outflow	Power
$\text{Conv}64-64$	16	22424	22424	114	240MHz	52μs	19230 images/s	3.7W
$4\times\text{Conv}64-64$	16	89746	75235	456	240MHz	208μs	19230 images/s	6.5W
$3\times\text{Conv}128-128$	64	134090	102552	171	240MHz	$103,2\mu$s	29069 images/s	7.8W
$3\times\text{Conv}256-256$	128	154599	102723	87	218MHz	$112,8\mu$s	26595 images/s	7.8W
$3\times\text{Conv}512-512$	128	132155	52151	45	208MHz	177μs	16949 images/s	7.9W

The processing unit uses $\mathbf{X}^2$ and a vector $\mathbf{W}$ made of P values coded on 1 bit each corresponding to weights in the convolution kernel. It thus computes in parallel P feature vectors (cf. Figure 2). The First-Input signal (FI) is set to 1 when the first feature vector is read from the second BRAM to initialise registers by 0. To compute each feature vector p where $1 \leq p \leq P$, we use the corresponding W_p to add either $\mathbf{X}^2$ or $\mathbf{-X}^2$ to the content of register p. Once all input feature vectors have been read from the second BRAM of memory block, the signal `Enable_s` is set to 1, and the content of registers is written one by one into the first BRAM of the memory block of the next layer. At the end of this process, the `Itter_done` signal is set to 1 in the processing unit block, so new data can be read from the memory block to process other feature vectors.

To achieve the computation associated with the layer block described in Figure 1, $k_{max}j_{max}$ clock cycles (CCs) are required to copy all contents from the first BRAM to the second one, $j_{max}k_{max}\ell_{max}/P$ CCs to compute all output feature vectors of one layer, and $j_{max}\ell_{max}$ CCs to write all computed feature vectors into the memory block of the next layer. Thus the total number of CCs required is:

$$CCs = j_{max}k_{max} + \frac{j_{max}k_{max}\ell_{max}}{P} + j_{max}\ell_{max}. \quad (2)$$

When comparing the total number of CCs of the proposed method with those obtained in [11], that are introduced by the following equation:

$$CCs = \frac{3j_{max}^2 k_{max}\ell_{max}}{P}, \quad (3)$$

we observe that the proposed architecture is $3j_{max}$ faster than [11], which can be significant when j_{max} is big. For instance with the CIFAR10 dataset, at the input layer of a CNN $j_{max} = 32$, and thus the proposed method is 96 times faster. In addition it is a pipeline architecture, so it can be $3Lj_{max}$ faster where L is the total number of layer blocks that fit in an FPGA.

We implemented one/few layers of Resnet18 on Xilinx Ultra Scale Vu13p (xcvu13p-figd2104-1-e) FPGA using VHDL. The implemented layers are arranged in a pipeline, and their functionality has been verified comparing the output of each layer block with the ones obtained by software simulation over a batch of examples. Table II shows the required resources to implement one/few layers of Resnet18 trained on CIFAR10 dataset for different values of P. It also shows that the obtained architecture yields a low processing latency to compute a valid output of one layer. Moreover, this processing latency increases when dealing with more than one layer, but the corresponding outflow is maintained thanks to the pipeline.

IV. CONCLUSION

In this paper, we proposed to reconsider the hardware implementation and acceleration of DNNs on limited resources embedded systems such as FPGA. We proposed to use lightweight DNNs architectures based on shift attention layers, and to combine them with weight binarization to reduce both complexity and memory usage. The resulting architecture almost matches the accuracy of considered baselines and only requires multiplexers, easing hardware implementation.

We implemented the proposed scheme using a low cost hardware architecture in which complex convolution operations are replaced by simple multiplexers. As a result, we were able to implement a considerable part of a complex CNNs (Resnet18). Moreover, the architecture only consumes a few watts and does not use any external ressources, making it a good solution for embedded applications. Future work will

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 06:46:50 UTC from IEEE Xplore. Restrictions apply.

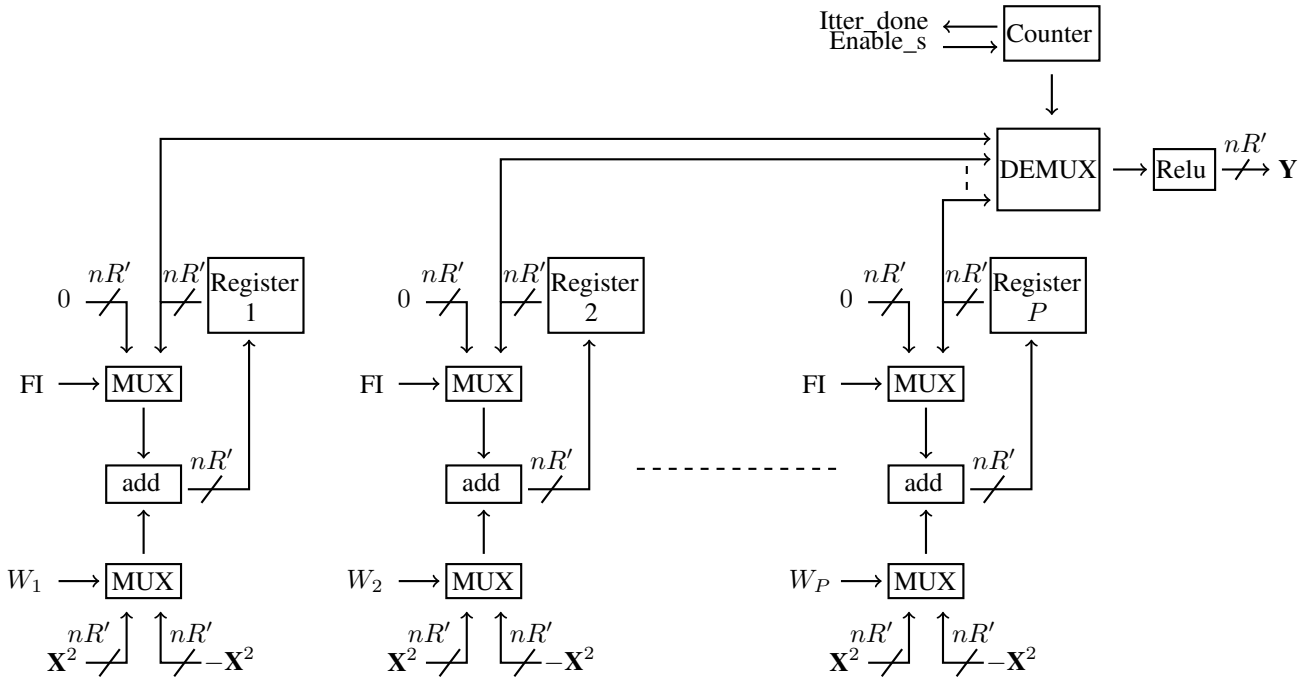


Fig. 2. Hardware architecture of a processing unit block.

extend this method to all kernel shapes, and propose a low cost hardware architecture to handle other challenging vision datasets such as ImageNet.

REFERENCES

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "Squeezenet: Alexnet-level accuracy with 50x fewer parameters and $<$ 0.5 mb model size," *arXiv preprint arXiv:1602.07360*, 2016.

[3] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *CoRR*, vol. abs/1409.1556, 2014. [Online]. Available: http://arxiv.org/abs/1409.1556

[4] B. Graham, "Fractional max-pooling," *CoRR*, vol. abs/1412.6071, 2014. [Online]. Available: http://arxiv.org/abs/1412.6071

[5] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," *arXiv preprint arXiv:1512.00567*, 2015.

[6] M. Courbariaux, Y. Bengio, and J.-P. David, "Binaryconnect: Training deep neural networks with binary weights during propagations," in *Advances in neural information processing systems*, 2015, pp. 3123–3131.

[7] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "Xnor-net: Imagenet classification using binary convolutional neural networks," in *European Conference on Computer Vision*. Springer, 2016, pp. 525–542.

[8] F. Li, B. Zhang, and B. Liu, "Ternary weight networks," *arXiv preprint arXiv:1605.04711*, 2016.

[9] S. Zhou, Y. Wu, Z. Ni, X. Zhou, H. Wen, and Y. Zou, "Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients," *arXiv preprint arXiv:1606.06160*, 2016.

[10] R. Andri, L. Cavigelli, D. Rossi, and L. Benini, "Yodann: An ultra-low power convolutional neural network accelerator based on binary weights." in *ISVLSI*, 2016, pp. 236–241.

[11] A. Ardakani, C. Condo, and W. J. Gross, "A convolutional accelerator for neural networks with binary weights," in *Circuits and Systems (ISCAS), 2018 IEEE International Symposium on*. IEEE, 2018, pp. 1–5.

[12] B. Wu, A. Wan, X. Yue, P. Jin, S. Zhao, N. Golmant, A. Gholaminejad, J. Gonzalez, and K. Keutzer, "Shift: A zero flop, zero parameter alternative to spatial convolutions," *arXiv preprint arXiv:1711.08141*, 2017.

[13] G. B. Hacene, C. Lassance, V. Gripon, M. Courbariaux, and Y. Bengio, "Attention based pruning for shift networks," *arXiv preprint arXiv:1905.12300*, 2019.

[14] J. Qiu, J. Wang, S. Yao, K. Guo, B. Li, E. Zhou, J. Yu, T. Tang, N. Xu, S. Song *et al.*, "Going deeper with embedded fpga platform for convolutional neural network," in *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. ACM, 2016, pp. 26–35.

[15] M. Blott, T. B. Preusser, N. J. Fraser, G. Gambardella, K. O'brien, Y. Umuroglu, M. Leeser, and K. Vissers, "Finn-r: An end-to-end deep-learning framework for fast exploration of quantized neural networks," *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, vol. 11, no. 3, p. 16, 2018.

[16] L. Jiao, C. Luo, W. Cao, X. Zhou, and L. Wang, "Accelerating low bit-width convolutional neural networks with embedded fpga," in *2017 27th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2017, pp. 1–4.

[17] K. Guo, S. Han, S. Yao, Y. Wang, Y. Xie, and H. Yang, "Software-hardware codesign for efficient neural network acceleration," *IEEE Micro*, vol. 37, no. 2, pp. 18–25, 2017.

[18] Y. Yang, Q. Huang, B. Wu, T. Zhang, L. Ma, G. Gambardella, M. Blott, L. Lavagno, K. Vissers, J. Wawrzynek *et al.*, "Synetgy: Algorithm-hardware co-design for convnet accelerators on embedded fpgas," in *Proceedings of the 2019 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. ACM, 2019, pp. 23–32.

[19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.

[20] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[21] S. Zagoruyko and N. Komodakis, "Wide residual networks," *arXiv preprint arXiv:1605.07146*, 2016.

[22] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks." in *CVPR*, vol. 1, no. 2, 2017, p. 3.

Authorized licensed use limited to: CSIR Madras Complex. Downloaded on May 29,2026 at 06:46:50 UTC from IEEE Xplore. Restrictions apply.